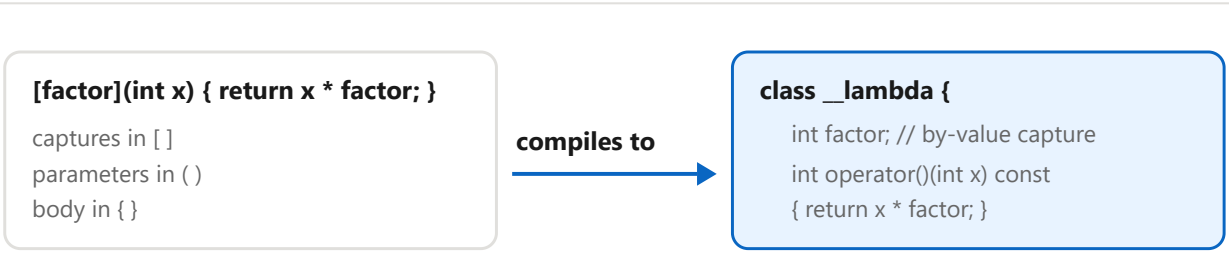
Lambdas

What a lambda is, how captures work and where they go wrong, init-capture, generic lambdas, and where lambdas are used. Assumes Function Pointers and Functors.

What a lambda is

A lambda is a function you write **right where you need it**, with no name. It has three parts: what it can see from outside (the **captures**), what it receives (the **parameters**), and what it does (the **body**).



The compiler turns the lambda into a small class: captures become members, the body becomes operator().

```
//  [captures] (parameters) -> return_type { body }
auto add = [](int a, int b) { return a + b; };
add(2, 3);                                          // 5

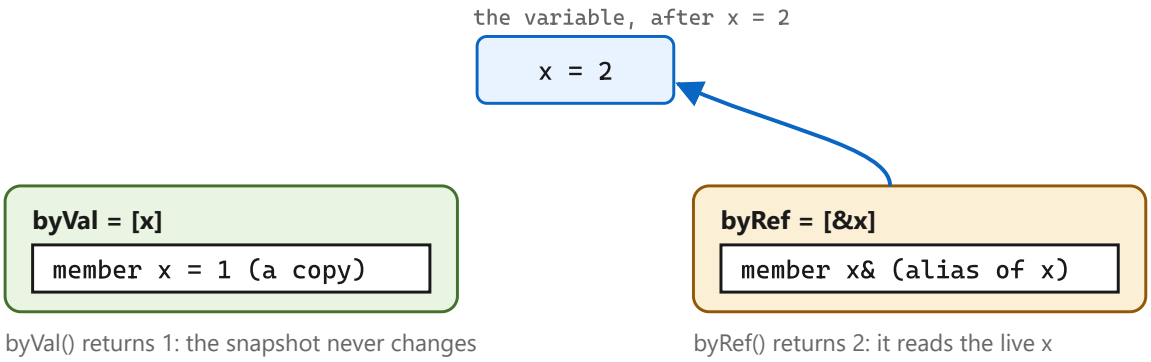
std::vector<int> v{5, 2, 9, 1};
// an inline comparator
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

- Every lambda has its **own unique type**, so store it with `auto` (or a template parameter).
- A lambda with **no captures** converts to a plain function pointer. Compiled: `sizeof` of a lambda with no captures is 1, of one capturing an `int` is 4, of a function pointer 8.
- `operator()` is `const` by default; `mutable` lifts that (see below).

WHERE IT COMES FROM A lambda is a functor that the compiler writes for you. If functors are new, read Function Pointers and Functors first.

Captures: by value and by reference

The square brackets say which outside variables the lambda may use, and how it holds them. **By value** copies the variable into the lambda when it is created. **By reference** keeps an alias to the live variable.



A by-value capture is a snapshot; a by-reference capture follows the variable.

```
int factor = 3;
// copies factor now
auto byValue = [factor](int x) { return x * factor; };
// refers to the live variable
auto byRef   = [&factor](int x) { return x * factor; };
// [=] all by value, [&] all by reference
auto all     = [=] (int x) { return x * factor; };
```

CAPTURE	MEANING
`[]`	nothing from outside
`[x]`	a copy of `x`
`[&x]`	a reference to `x`
`[=]`	copy everything the lambda uses
`[&]`	reference everything the lambda uses
`[x = expr]`	a new member initialized from `expr` (see below)

PREFER NAMING THEM A list such as `[limit, &total]` shows at a glance what the lambda depends on. `[=]` and `[&]` hide it.

Capture lifetime: dangling references

A by-reference capture is only valid while the variable lives. If the lambda **outlives** it, because it is returned, stored, or run later on another thread, the reference points at a destroyed variable.



calling the closure now reads memory that no longer belongs to n

The closure left the function; the local it refers to did not.

```cpp
auto make() {
    int n = 5;
    return [&n] { return n; };      // BUG: n dies when make() returns
}

auto makeSafe() {
    int n = 5;
    return [n] { return n; };       // fine: the lambda owns its own copy
}
```

THE LAMBDA IS...	CAPTURE BY REFERENCE IS...
used and finished inside the same scope (`std::sort`, `count_if`)	safe
returned or stored for later	**dangling**: capture by value
run on another thread that may outlive the variable	**dangling**: capture by value

> `[=]` AND `THIS` Inside a member function `[=]` also captures `this` (the pointer), so the lambda uses the object through that pointer and dangles if the object dies. This implicit capture is deprecated in C++20: write `[=, this]` to say so, or `[*this]` to copy the object.

`mutable`, init-capture, generic and recursive lambdas

`mutable`: the lambda's own state

```cpp
// n is the lambda's own member
auto count = [n = 0]() mutable { return ++n; };
// 1 2 3 (compiled: "123")
count(); count(); count();
```

A by-value capture is `const` inside the lambda; `mutable` lets it change. The change lives in the lambda's own copy, never in the original variable, and each copy of the lambda has its own state.

Init-capture: move something in



A move-only object cannot be copied into a lambda, but it can be moved in.

```
auto owned = std::make_unique<int>(42);
// C++14 init-capture
auto init  = [p = std::move(owned)] { return *p; };
// 42; owned is now empty
init();
```

`[name = expr]` creates a new member initialized from `expr`. It is the way to move a `unique_ptr` in, or to precompute a value once.

Generic lambdas

```
auto bigger = [](const auto& a, const auto& b) { return a < b ? b : a; };
bigger(3, 7);                                                  // 7
bigger(2.5, 1.5);                                              // 2.5
```

With `auto` parameters (C++14) the lambda's `operator()` is a template, so one lambda works for any types that support `<`.

Recursion and immediate calls

```
auto fact = [](auto self, int n) -> int { return n ≤ 1 ? 1 : n * self(self,
n - 1); };
// 120: a lambda cannot name itself, so it is passed to itself
fact(fact, 5);

// called at once: initialize a const with logic
const int limit = [&] { return fast ? 100 : 10; }();
```

Where lambdas are used

Almost everywhere something takes a small piece of behavior.

SITUATION	EXAMPLE
sort with your own order	`std::sort(w.begin(), w.end(), [](const std::string& a, const std::string& b) { return a.size() < b.size(); });`
find or count by a rule	`std::find_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });`
a task for a thread	`std::thread t([&] { work(); });` (capture carefully, see lifetime)
a callback you keep for later	`std::function<void()> onDone = [id] { notify(id); };`
initialize a `const` with logic	`const auto v = [&] { ... }();`

Compiled: sorting `{"pear", "fig", "banana"}` with the length comparison above put `fig` first and `banana` last.

> **RULE OF THUMB** If the logic is more than a few lines, or you use it in several places, give it a name (a function) instead of a long lambda.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
What a lambda is	C++ Primer	10.3 Customizing operations	414 to 429
	C++ Primer	14.8 The function-call operator	600 to 607
Captures: by value and by reference	C++ Primer	10.3 Customizing operations	414 to 429
mutable, init-capture, generic and recursive lambdas	Programming: Principles and Practice	21.1 to 21.2, 21.5 Standard algorithms, function objects, sorting and searching	617 to 626 and 633 to 635
	Effective Modern C++	Item 31, Avoid default capture modes	234 to 242
	Effective Modern C++	Item 32, Use init capture to move objects into closures	242 to 247